

DOCUMENTATION COMPLETE

Serveur d'examen local avec Raspberry Pi

Application d'examen numérique sans Internet

Backend Node.js / Express - Frontend React - SQLite - PM2 - Wi-Fi local

Adresse finale en salle : <http://192.168.4.1:8000> | Etudiants : [/student/login](#) | Formateur : [/trainer/login](#)

Version stable - Documentation technique et fonctionnelle

Table des matières

1. 1. Présentation du projet
2. 2. Objectifs fonctionnels
3. 3. Architecture générale
4. 4. Technologies utilisées
5. 5. Base de données SQLite
6. 6. Configuration backend
7. 7. Routes API backend
8. 8. Frontend React
9. 9. Sécurité applicative
10. 10. Build et exécution locale
11. 11. Déploiement sur Raspberry Pi
12. 12. Démarrage automatique avec PM2
13. 13. Wi-Fi local sans Internet
14. 14. Procédure d'utilisation en salle
15. 15. Maintenance
16. 16. Problèmes fréquents et solutions
17. 17. Checklist finale
18. 18. Conclusion

1. Présentation du projet

Le projet consiste à créer une application d'examen numérique fonctionnant localement sur un Raspberry Pi, sans dépendre d'Internet. Le Raspberry Pi joue le rôle de serveur backend, serveur frontend, base de données locale SQLite et point d'accès Wi-Fi local.

L'objectif est de permettre à un formateur de créer des examens QCM, de gérer les étudiants autorisés, de démarrer une session d'examen et de suivre les résultats. Les étudiants se connectent au réseau local du Raspberry Pi et passent l'examen depuis leur navigateur.

Le système est conçu pour les universités, écoles de formation et centres d'examen souhaitant organiser des tests numériques en salle sans connexion Internet.

2. Objectifs fonctionnels

2.1 Côté formateur

- Se connecter à l'espace formateur
- Gérer les modules, examens, questions et réponses
- Gérer les étudiants autorisés et les formateurs
- Démarrer et fermer une session d'examen
- Consulter les résultats et les détails de tentative
- Voir les événements de sécurité
- Suivre les statistiques depuis un dashboard

2.2 Côté étudiant

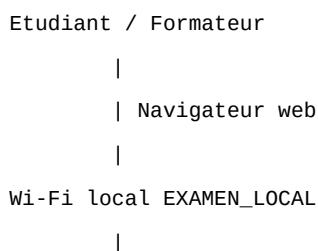
- Se connecter avec matricule et code secret
- Choisir un examen publié
- Accéder uniquement si la session est ouverte
- Répondre aux questions avec chrono
- Soumettre l'examen
- Voir son résultat

2.3 Objectifs de sécurité

- Limiter l'accès aux étudiants autorisés
- Bloquer les étudiants désactivés
- Détecter la sortie de fenêtre pendant l'examen
- Arrêter une tentative en cas de comportement suspect
- Hasher les mots de passe formateurs avec bcrypt
- Ne pas exposer password_hash côté frontend
- Utiliser une base SQLite locale

3. Architecture générale du projet

3.1 Architecture globale



| http://192.168.4.1:8000

|

Raspberry Pi

|

- +-- Frontend React buildé
- +-- Backend Node.js / Express
- +-- Base SQLite locale
- +-- PM2 pour démarrage automatique
- +-- Point d'accès Wi-Fi local

3.2 Architecture des dossiers

exam-server/

```

├─ backend/
|   ├─ config/db.js
|   ├─ database/exam_server.db
|   ├─ routes/
|       ├─ moduleRoute.js
|       ├─ examRoute.js
|       ├─ questionRoute.js
|       ├─ answerRoute.js
|       ├─ studentRoute.js
|       ├─ studentAdminRoute.js
|       ├─ trainerRoute.js
|       └─ trainerAdminRoute.js
|   ├─ server.js
|   ├─ package.json
|   └─ .env
└─ frontend/
    ├─ src/
    |   ├─ api/api.js
    |   ├─ pages/
    |   ├─ layouts/
    |   ├─ routes/
    |   ├─ App.jsx
    |   └─ main.jsx
    ├─ dist/
    └─ package.json

```

4. Technologies utilisées

Technologie	Rôle
Node.js	Environnement JavaScript côté serveur
Express.js	Création de l'API REST
SQLite	Base de données locale embarquée

better-sqlite3	Connexion Node.js - SQLite
bcrypt	Hashage des mots de passe formateurs
dotenv	Variables d'environnement
PM2	Démarrage automatique du serveur
React	Interface utilisateur
Vite	Build du frontend
Material UI	Composants graphiques
Axios	Appels API
React Router DOM	Routes frontend
NetworkManager / nmcli	Wi-Fi local et IP fixe

5. Base de données SQLite

La base de données principale est stockée dans :

`backend/database/exam_server.db`

Table	Description
trainers	Comptes formateurs
students	Etudiants autorisés
modules	Modules pédagogiques
exams	Examens
questions	Questions des examens
answers	Réponses possibles
exam_sessions	Sessions ouvertes/fermées
exam_attempts	Tentatives des étudiants
attempt_questions	Questions attribuées à une tentative
student_answers	Réponses données par l'étudiant
security_events	Evénements de sécurité

6. Configuration backend

6.1 Fichier .env

```
PORT=8000
DB_PATH=./database/exam_server.db
NODE_ENV=production
```

6.2 Connexion SQLite

```
import Database from "better-sqlite3";
import dotenv from "dotenv";

dotenv.config();

const dbPath = process.env.DB_PATH || "./database/exam_server.db";
const db = new Database(dbPath);
db.pragma("foreign_keys = ON");
```

```
export default db;
```

7. Routes API backend

Modules

Base : /api/modules

Méthode	Endpoint	Description
GET	/api/modules	Lister les modules
GET	/api/modules/:id	Voir un module
POST	/api/modules	Créer un module
PUT	/api/modules/:id	Modifier un module
DELETE	/api/modules/:id	Supprimer un module

Examens

Base : /api/exams

Méthode	Endpoint	Description
GET	/api/exams	Lister les examens
GET	/api/exams/:id	Voir un examen
POST	/api/exams	Créer un examen
PUT	/api/exams/:id	Modifier un examen
DELETE	/api/exams/:id	Supprimer un examen

Questions

Base : /api/questions

Méthode	Endpoint	Description
GET	/api/questions	Lister les questions
GET	/api/questions/:id	Voir une question
GET	/api/questions/exam/:exam_id	Questions d'un examen
POST	/api/questions	Créer une question
PUT	/api/questions/:id	Modifier
DELETE	/api/questions/:id	Supprimer

Réponses

Base : /api/answers

Méthode	Endpoint	Description
GET	/api/answers	Lister les réponses
GET	/api/answers/:id	Voir une réponse
GET	/api/answers/question/:question_id	Réponses d'une question
POST	/api/answers	Créer une réponse
PUT	/api/answers/:id	Modifier
DELETE	/api/answers/:id	Supprimer

Etudiants - administration

Base : /api/admin/students

Méthode	Endpoint	Description
GET	/api/admin/students	Lister
POST	/api/admin/students	Ajouter
PUT	/api/admin/students/:id	Modifier
PATCH	/api/admin/students/:id/toggle	Activer/désactiver
DELETE	/api/admin/students/:id	Supprimer

Formateurs - administration

Base : /api/admin/trainers

Méthode	Endpoint	Description
GET	/api/admin/trainers	Lister
POST	/api/admin/trainers	Ajouter
PUT	/api/admin/trainers/:id	Modifier
DELETE	/api/admin/trainers/:id	Supprimer

Etudiant et passage examen

Base : /api/student

Méthode	Endpoint	Description
POST	/api/student/login	Connexion étudiant
POST	/api/student/start-exam	Démarrer tentative
POST	/api/student/submit-exam	Soumettre
GET	/api/student/exam-timer/:attempt_id	Vérifier le chrono
POST	/api/student/security-event	Signaler événement sécurité

Sessions d'examen

Base : /api/exam-session

Méthode	Endpoint	Description
POST	/api/exam-session/start	Démarrer session
POST	/api/exam-session/close	Fermer session
GET	/api/exam-session/open/:exam_id	Vérifier session ouverte

Résultats

Base : /api/trainer/results

Méthode	Endpoint	Description
GET	/api/trainer/results	Tous les résultats
GET	/api/trainer/results/exam/:exam_id	Résultats d'un examen
GET	/api/trainer/results/attempt/:attempt_id	Détail d'une tentative

7.1 Exemples de requêtes JSON

Créer un examen :

```
{
  "module_id": 1,
  "trainer_id": 1,
  "title": "Examen SQLite",
  "description": "QCM sur SQLite",
  "duration_minutes": 30,
  "number_of_questions": 5,
  "success_percentage": 60,
  "random_questions": 1,
  "random_answers": 1,
  "is_published": 1
}
```

Créer un étudiant :

```
{
  "matricule": "ETU001",
  "first_name": "Aminata",
  "last_name": "Diop",
  "secret_code": "1111",
  "is_active": 1
}
```

Démarrer un examen étudiant :

```
{
  "matricule": "ETU001",
  "secret_code": "1111",
  "exam_id": 1
}
```

8. Frontend React

8.1 Configuration API

```
import axios from "axios";

const api = axios.create({
  baseURL: "/api",
});

export default api;
```

La configuration `baseURL: "/api"` est indispensable pour que l'application fonctionne depuis les autres machines du réseau

local.

8.2 Routes frontend

Route	Page
/	Redirection vers /student/login
/student/login	Connexion étudiant
/student/exam	Passage de l'examen
/student/result	Résultat étudiant
/trainer/login	Connexion formateur
/trainer/dashboard	Dashboard formateur
/trainer/modules	Gestion modules
/trainer/exams	Gestion examens
/trainer/students	Gestion étudiants
/trainer/trainers	Gestion formateurs
/trainer/questions	Gestion questions/réponses
/trainer/results	Résultats examens

9. Sécurité applicative

9.1 Formateurs

- Les mots de passe sont hashés avec bcrypt
- password_hash n'est jamais envoyé au frontend
- Si le champ mot de passe est vide en modification, l'ancien mot de passe est conservé
- Si un nouveau mot de passe est saisi, un nouveau hash est généré

9.2 Etudiants

- Matricule obligatoire
- Code secret obligatoire
- Compte étudiant actif obligatoire
- Examen publié obligatoire
- Session ouverte obligatoire

9.3 Pendant l'examen

- Chrono automatique
- Questions et réponses aléatoires selon configuration
- Détection de sortie de fenêtre
- Enregistrement des événements de sécurité
- Résultat enregistré automatiquement

10. Build et exécution locale

10.1 Installer les dépendances

```
cd /home/tidiane/exam-server/backend
npm install
```

```
cd /home/tidiane/exam-server/frontend
npm install
```

10.2 Build frontend

```
cd /home/tidiane/exam-server/frontend
npm run build
```

10.3 Lancer le serveur

```
cd /home/tidiane/exam-server/backend
node server.js
```

Accès	URL
Local Raspberry	http://localhost:8000
Réseau local	http://IP_DU_RASPBERRY:8000
Wi-Fi local autonome	http://192.168.4.1:8000

11. Déploiement sur Raspberry Pi

Emplacement recommandé :

```
/home/tidiane/exam-server
sudo apt update
sudo apt install nodejs npm sqlite3 -y
```

```
cd /home/tidiane/exam-server/backend
npm install
```

```
cd /home/tidiane/exam-server/frontend
npm install
npm run build
```

```
cd /home/tidiane/exam-server/backend
node server.js
```

12. Démarrage automatique avec PM2

```
sudo npm install -g pm2
cd /home/tidiane/exam-server/backend
pm2 start server.js --name exam-server
pm2 list
pm2 startup
# Copier-coller la commande sudo env PATH=... affichée par PM2
pm2 save
sudo reboot
```

Commande	Rôle
pm2 list	Voir les applications
pm2 logs exam-server	Voir les logs

pm2 restart exam-server	Redémarrer
pm2 stop exam-server	Arrêter
pm2 save	Sauvegarder l'état

13. Wi-Fi local sans Internet

13.1 Objectif

Elément	Valeur
Nom Wi-Fi	EXAMEN_LOCAL
Mot de passe	examen2026
IP Raspberry	192.168.4.1
Application	http://192.168.4.1:8000

13.2 Créer le hotspot

```
sudo nmcli device wifi hotspot ifname wlan0 ssid EXAMEN_LOCAL password examen2026
```

13.3 Fixer l'adresse IP du hotspot

```
nmcli connection show
sudo nmcli connection modify Hotspot ipv4.addresses 192.168.4.1/24
sudo nmcli connection modify Hotspot ipv4.method shared
sudo nmcli connection down Hotspot
sudo nmcli connection up Hotspot
sudo nmcli connection modify Hotspot connection.autoconnect yes
sudo reboot
```

13.4 Adresse finale

Utilisateur	Adresse
Etudiants	http://192.168.4.1:8000/student/login
Formateur	http://192.168.4.1:8000/trainer/login
Par défaut	http://192.168.4.1:8000

14. Procédure d'utilisation en salle

14.1 Avant l'examen

- Allumer le Raspberry
- Vérifier que le Wi-Fi EXAMEN_LOCAL est visible
- Se connecter à l'espace formateur
- Créer ou vérifier module, examen, questions et réponses
- Vérifier que l'examen est publié
- Ajouter ou vérifier les étudiants autorisés
- Démarrer la session d'examen

14.2 Pendant l'examen

- Les étudiants se connectent au Wi-Fi EXAMEN_LOCAL
- Ils ouvrent `http://192.168.4.1:8000`
- Ils saisissent matricule et code secret
- Ils choisissent l'examen
- Ils répondent et soumettent

14.3 Après l'examen

- Fermer la session
- Consulter les résultats
- Voir les scores et validations
- Contrôler les tentatives arrêtées
- Analyser les événements de sécurité
- Sauvegarder la base SQLite si nécessaire

15. Maintenance

Cas	Commande
Modification frontend	<code>cd /home/tidiane/exam-server/frontend && npm run build && pm2 restart exam-server</code>
Modification backend	<code>cd /home/tidiane/exam-server/backend && pm2 restart exam-server</code>
Sauvegarde base	<code>cp /home/tidiane/exam-server/backend/database/exam_server.db /home/tidiane/exam_server_backup.db</code>
Voir logs	<code>pm2 logs exam-server</code>

16. Problèmes fréquents et solutions

Problème	Cause probable	Solution
La liste des examens ne s'affiche pas sur les autres machines	baseURL axios configurée avec localhost	Mettre baseURL: <code>"/api"</code> , refaire <code>npm run build</code> , redémarrer PM2
Le serveur ne démarre pas	Erreur backend ou dépendance manquante	<code>pm2 logs exam-server</code> puis <code>node server.js</code>
La base SQLite ne fonctionne pas	Mauvais chemin <code>DB_PATH</code> ou fichier absent	Vérifier <code>backend/database/exam_server.db</code> et <code>.env</code>
Dashboard vide	Route résultats ou jointure SQL incorrecte	Tester <code>/api/trainer/results</code> et vérifier <code>exam_attempts.session_id -> exam_sessions.exam_id</code>
Wi-Fi local absent	Hotspot non démarré	<code>nmcli connection show</code> puis <code>nmcli connection up Hotspot</code>
PM2 ne démarre pas au boot	startup non enregistré	<code>pm2 startup</code> puis <code>pm2 save</code>

17. Checklist finale

- ☐ Raspberry allumé
- ☐ Wi-Fi EXAMEN_LOCAL visible
- ☐ Application accessible sur `http://192.168.4.1:8000`
- ☐ PM2 indique exam-server online
- ☐ Formateur peut se connecter
- ☐ Étudiants autorisés créés
- ☐ Examen publié
- ☐ Questions et réponses vérifiées
- ☐ Session démarrée

- ☐ Étudiant test peut commencer
- ☐ Résultat enregistré

18. Conclusion

Cette application permet de réaliser des examens numériques localement, sans connexion Internet, avec un Raspberry Pi comme serveur autonome. Elle couvre la gestion des examens, des étudiants, des formateurs, des questions, des réponses, des sessions, du chrono, de la sécurité et des résultats.

Adresse finale à retenir : <http://192.168.4.1:8000>

Cette configuration est adaptée aux universités, écoles de formation et centres d'examen souhaitant organiser des tests numériques en salle sans dépendre d'Internet.